# Last Deep Learning Details

Extended deep learning notes Sources: listed under References

### What this file is really about:

$$\text{names on the slides} \rightarrow \text{what each one changes} \rightarrow \text{the calculation behind it}$$

**How to use this file.** A final audit compared every slide title against this course. Each section here covers a model or method that was named in the lectures and not yet explained. Part B has checked code.

### Step-up path

**You need.** The deep learning architecture, transformer and generative sections.

1. Compute a causal receptive field.
2. Compare attention scores.
3. Choose a box loss.
4. Diagnose posterior collapse.
5. Reason about roofline limits and batching.

**You can stop when** you can explain each model named on the slides.

---

## Contents

# Part A Maths From the Ground Up

## A1 The Elman network and truncated backpropagation through time

The simplest recurrent network, due to Elman, is

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b), \qquad \hat{y}_t = W_y h_t.$$

The full gradient runs back through every step, which costs memory proportional to the sequence length. **Truncated BPTT** detaches the hidden state every $k$ steps. It keeps the state but ignores dependencies longer than $k$ when computing the gradient, so a model can carry information further than it can learn to use.

---

## A2 Additive and multiplicative attention

For a decoder state $s$ and encoder state $h_i$, attention needs a score. Softmax then gives weights, and the context is their weighted sum.

| Name | Score |
|---|---|
| Bahdanau (additive) | $v^\top \tanh(W_s s + W_h h_i)$ |
| Luong dot | $s^\top h_i$ |
| Luong general | $s^\top W h_i$ |
| Scaled dot product | $s^\top h_i / \sqrt{d}$ |

Additive attention has its own small network, and dot products need only a matrix multiplication, so they run much faster. The $\sqrt{d}$ scale stops large dimensions pushing the softmax into saturation.

---

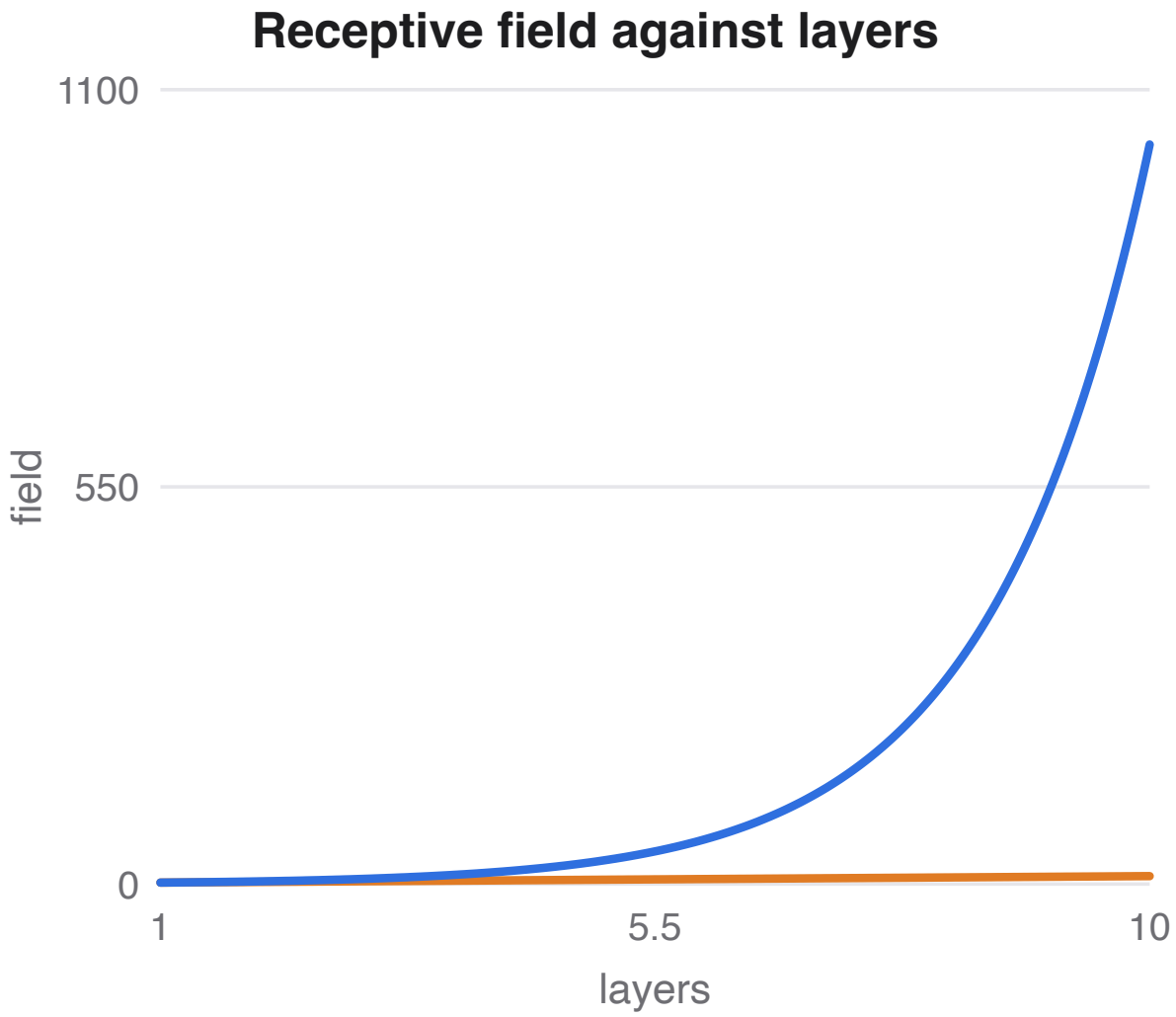
## A3 Causal convolution, WaveNet and the TCN

A convolution is **causal** if the output at time $t$ uses only inputs at or before $t$. With left padding of $(f-1)d$, the length is preserved. Stacking dilations $1, 2, 4, \ldots, 2^{L-1}$ with kernel 2 gives a receptive field of $2^L$.

**WaveNet** uses gated activation blocks in place of ReLU:

$$z = \tanh(W_f * x) \odot \sigma(W_g * x).$$

The sigmoid acts as a learned gate on the tanh features. Residual and skip connections let many layers train. A **TCN** is a simplified version: dilated causal convolutions with residual blocks and weight normalisation. Unlike a recurrent network, every step in a TCN can run in parallel.




---

## A4 Strict and inclusive causal masks

An autoregressive model predicts each element from the earlier ones. A convolution or attention mask may include the current position or not.

- A **strict** mask, called type A in PixelCNN, excludes the current element. It is needed in the first layer, which sees the raw value being predicted.

- An **inclusive** mask, type B, allows the current position. It is safe in later layers because that position already contains only allowed information.

---

## A5 BERT, BART, GPT-2 and T5

**BERT** pretrains by masked language modelling: about 15% of tokens are hidden, and the encoder predicts them with cross-entropy over the masked positions only. A segment embedding and special tokens support pairs of sentences.

**BART** is a denoising encoder-decoder. The input is corrupted by masks, deletions and shuffling, and the decoder reconstructs the original. That suits summarisation.

**GPT-2** is a scaled decoder-only model with one striking claim: a large language model can do several tasks unsupervised when prompted correctly.

**T5** treats every task as text to text, using a task prefix such as "summarize:".

**Extractive QA** does not write an answer. It predicts a start and an end position inside a passage, and the best span maximises $s_i + e_j$ subject to $i \leq j$ and a length limit.

---

## A6 Tokenisers and small adapters

**WordPiece** splits an unknown word greedily into the longest known pieces, marking continuations. **SentencePiece** is a library that learns its subwords directly on raw text, without assuming spaces, so it suits many languages.

**Prefix tuning** freezes a model and learns a continuous set of prefix vectors added before each layer's attention. **LoRA** instead learns a low-rank weight update. Both are parts of the broader PEFT family, which train only a small share of the parameters.

---

## A7 Vision: DeiT, hierarchy and vision-language links

**DeiT** trains a transformer on ImageNet without vast data, using strong augmentation and a teacher through a distillation token. A **hierarchical ViT** such as Swin merges patches as it goes deeper and limits attention to shifted windows, so it gives multi-scale features for detection.

**BLIP-2** freezes an image encoder and a language model, and trains only a small Q-Former that connects the two. **SigLIP** replaces CLIP's softmax with a sigmoid per image-text pair, so batches need not be fully

contrastive.

---

## A8 More self-supervision

**Multi-crop** gives a few large and several small views, and trains the small views to match the large ones. It is a key part of DINO. **Barlow Twins** pushes the cross-correlation matrix $C$ between two views towards the identity:

$$L = \sum_i (1 - C_{ii})^2 + \lambda \sum_{i \neq j} C_{ij}^2.$$

**VICReg** uses three explicit terms: invariance between views, variance above a floor in each dimension, and low covariance between dimensions. Both avoid collapse without negatives.

---

## A9 Augmentation and box losses

**RandAugment** samples $N$ operations at one shared magnitude $M$. **TrivialAugment** goes further: one random operation at a random strength, with no tuning.

Box regression losses address different failures. **Smooth L1** is quadratic near zero and linear for large errors, so outliers do not dominate:

$$\mathrm{SmoothL1}(x) = \begin{cases} \tfrac{1}{2} x^2 / \beta & |x| < \beta \\ |x| - \tfrac{1}{2}\beta & \text{otherwise.} \end{cases}$$

IoU has no gradient when the boxes do not overlap. With $C$ the smallest box enclosing both:

$$\mathrm{GIoU} = \mathrm{IoU} - \frac{|C \setminus (A \cup B)|}{|C|}, \qquad \mathrm{DIoU} = \mathrm{IoU} - \frac{\rho^2}{c^2}.$$

Here $\rho$ is the distance between centres and $c$ is the diagonal of $C$. **CIoU** adds a penalty for mismatched aspect ratio. The loss is 1 minus the score.

**SSD** predicts from several feature maps at once, using default boxes. **RetinaNet** pairs a feature pyramid with focal loss, showing that the class imbalance, and not the single-stage design, had held accuracy back.

## A10 Problems and controls in VAEs

**Posterior collapse** happens when $q(z \mid x) \approx p(z)$, so the KL term is zero and the decoder ignores $z$. A powerful autoregressive decoder makes this more likely. Common fixes are KL annealing and a minimum KL per dimension, called free bits.

A **latent traversal** changes one coordinate of $z$ while holding the rest fixed, and decodes each result. The **$\beta$-VAE** weights the KL by $\beta > 1$, trading reconstruction for more independent factors:

$$L = \mathbb{E}_q[\log p(x \mid z)] - \beta \, \mathrm{KL}(q \| p).$$

## A11 GAN design and evaluation

**DCGAN** gave architecture rules that made training stable: strided convolutions in place of pooling, batch norm, ReLU in the generator and LeakyReLU in the discriminator. A **conditional GAN** feeds a label to both networks. **pix2pix** conditions on an image, and adds an L1 term to a patch discriminator.

**Fréchet Inception Distance** fits normals to Inception features of real and generated images:

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}\big(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\big).$$

Lower is better, and it depends on sample size. **Inception Score** is $\exp\left(\mathbb{E}_x \mathrm{KL}(p(y \mid x) \| p(y))\right)$, which rewards confident and varied labels but never looks at real images.

## A12 Controlling diffusion

**Inpainting** replaces the known region at each step with a suitably noised copy of the true image, and lets the model fill the rest. **ControlNet** copies the encoder of the denoiser, feeds it a condition such as edges, and adds its output through zero-initialised layers, so training starts as a no-op. **DreamBooth** finetunes the model on a few photos of one subject. **Textual inversion** freezes the model and learns only a new word embedding.

**Rectified flow** trains straight paths, $x_t = (1 - t)x_0 + t x_1$, with velocity target $x_1 - x_0$. Refitting on its own couplings, called reflow, straightens paths further so that few steps are enough. **Glow** is a flow with invertible $1 \times 1$ convolutions, whose log-determinant is cheap.

## A13 Hardware limits when serving

The **roofline** picture says a step's time is the greater of two limits:

$$t = \max\left(\frac{\text{FLOPs}}{\text{compute rate}}, \frac{\text{bytes moved}}{\text{memory bandwidth}}\right).$$

When the second term wins, a chip's peak FLOPs do not help. Language model decoding is usually memory-bound.

Serving engines allocate cache memory for key and value tensors in **pages** that need not be contiguous, so a new request does not reserve its maximum length up front. **Continuous batching** adds and removes requests at each step, and the throughput gain over waiting for a whole batch grows with the variation in lengths. vLLM combines both. A **TPU** is an accelerator built around a large systolic array for matrix multiplication.

**Mixture of experts** routes each token to a few sub-networks, so parameters grow faster than the work per token. With $E$ experts and $k$ active, per-token cost is about $k/E$ of the total expert size.

## A14 Splitting a model across devices

| Method | What is split | Main cost |
|---|---|---|
| Data parallel | Batch | Limited by model size |
| Model parallel | Layers across devices | Idle devices |
| Pipeline parallel | Layers, with overlapping minibatches | Stage balance |
| Tensor parallel | Matrices within each layer | Communication each layer |

**Gradient checkpointing** stores activations at only some layers and recomputes the rest during the backward pass. Memory falls from $O(n)$ to about $O(\sqrt{n})$ for at most a small constant more compute.

## A15 How models act and grow

**Function calling** lets a model emit a structured request. The application runs it, and returns the result to the model as text. **Constitutional AI** replaces some human labels with a model critiquing and revising its

own responses against written principles.

**Scaling laws** model loss as a power law of parameters, data and compute. The Chinchilla analysis found about 20 tokens per parameter was efficient for its loss. **Emergent abilities** are skills that appear abruptly with scale, but some of that abruptness comes from a metric's thresholds, so such claims should be stated carefully.

# Part B In Code

## B1 Causal convolution and its reach

```
import torch, torch.nn as nn
class Causal(nn.Module):
    def __init__(self, dilation):
        super().__init__(); self.pad = dilation; self.c = nn.Conv1d(1, 1, 2, dilation=dilat
    def forward(self, x): return self.c(nn.functional.pad(x, (self.pad, 0)))
net = nn.Sequential(*[Causal(2**i) for i in range(4)])
x = torch.zeros(1, 1, 40, requires_grad=True)
y = net(x); assert y.shape == x.shape
y[0, 0, 30].backward()
used = (x.grad[0, 0] != 0).nonzero().flatten()
print('first', used.min().item(), 'last', used.max().item(), 'reach', used.max().item() - u
assert used.max() == 30 and used.max() - used.min() + 1 == 16
```

## B2 Box losses when boxes do not overlap

```
def scores(a, b):
    ix = max(0, min(a[2], b[2]) - max(a[0], b[0])); iy = max(0, min(a[3], b[3]) - max(a[1],
    inter = ix * iy; area = lambda r: (r[2]-r[0]) * (r[3]-r[1]); union = area(a) + area(b)
    c = (min(a[0], b[0]), min(a[1], b[1]), max(a[2], b[2]), max(a[3], b[3]))
    iou = inter / union; giou = iou - (area(c) - union) / area(c)
    rho2 = ((a[0]+a[2]-b[0]-b[2])/2)**2 + ((a[1]+a[3]-b[1]-b[3])/2)**2
    diag2 = (c[2]-c[0])**2 + (c[3]-c[1])**2
    return iou, giou, iou - rho2 / diag2
near = scores((0,0,2,2), (3,0,5,2)); far = scores((0,0,2,2), (8,0,10,2))
print(near, far)
assert near[0] == far[0] == 0 and near[1] > far[1] and near[2] > far[2]
assert abs(scores((0,0,2,2), (0,0,2,2))[1] - 1) < 1e-12
```

## B3 FID between two Gaussians

```
import numpy as np
from scipy.linalg import sqrtm
def fid(m1, S1, m2, S2):
    c = sqrtm(S1 @ S2).real
    return float(np.sum((m1 - m2)**2) + np.trace(S1 + S2 - 2 * c))
I = np.eye(2)
print(round(fid(np.zeros(2), I, np.ones(2), I), 6), round(fid(np.zeros(2), I, np.zeros(2),
assert abs(fid(np.zeros(2), I, np.ones(2), I) - 2) < 1e-9
assert abs(fid(np.zeros(2), I, np.zeros(2), 4 * I) - 2) < 1e-9
```

## B4 Barlow Twins sees a collapsed embedding

```
import torch
def barlow(a, b, lam=0.005):
    a = (a - a.mean(0)) / (a.std(0) + 1e-9); b = (b - b.mean(0)) / (b.std(0) + 1e-9)
    C = a.T @ b / len(a); I = torch.eye(C.shape[0])
    return ((C.diagonal() - 1)**2).sum() + lam * ((C - I)**2 * (1 - I)).sum()
torch.manual_seed(0)
good = torch.randn(4000, 8)
collapsed = good[:, :1].repeat(1, 8)
print(round(barlow(good, good).item(), 4), round(barlow(collapsed, collapsed).item(), 4))
assert barlow(good, good) < barlow(collapsed, collapsed)
```

## B5 Continuous batching against static batching

```
import numpy as np
rng = np.random.default_rng(0)
lengths = rng.integers(10, 400, 2000)
B = 16
static = sum(lengths[i:i+B].max() for i in range(0, len(lengths), B))
slots = np.zeros(B, dtype=int); steps = 0; q = 0
while q < len(lengths) or slots.any():
    for i in range(B):
        if slots[i] == 0 and q < len(lengths): slots[i] = lengths[q]; q += 1
    slots = np.maximum(slots - 1, 0); steps += 1
print('static steps', static, 'continuous steps', steps, 'gain', round(static / steps, 2))
assert steps < static
```

## B6 Gradient checkpointing gives the same gradient

```
import torch, torch.nn as nn
from torch.utils.checkpoint import checkpoint
torch.manual_seed(0)
blocks = nn.ModuleList(nn.Sequential(nn.Linear(16, 16), nn.Tanh()) for _ in range(6))
x = torch.randn(4, 16)
def run(use):
    for p in blocks.parameters(): p.grad = None
    h = x
    for b in blocks: h = checkpoint(b, h, use_reentrant=False) if use else b(h)
    h.sum().backward(); return [p.grad.clone() for p in blocks.parameters()]
for a, b in zip(run(False), run(True)): assert torch.allclose(a, b)
print('checkpointed gradients match')
```

## Part C Practice

**P1.** A causal stack with kernel 2 uses dilations $1, 2, 4, 8, 16$. What is its receptive field?

$1 + 1 + 2 + 4 + 8 + 16 = 32.$

**P2.** Two boxes do not overlap. What does a plain IoU loss's gradient do, and what fixes it?

The gradient is zero. GIoU or DIoU give a useful signal, by depending on the enclosing box or the centre distance.

**P3.** Why is a strict mask needed only in the first layer?

Only the first layer sees the raw value being predicted. After that, the feature at a position contains only allowed context.

**P4.** What does a KL near zero in every dimension of a VAE mean?

Posterior collapse. The latent carries no information, and the decoder ignores it.

**P5.** FID between $\mathcal{N}(0, I)$ and $\mathcal{N}(0, 4I)$ in two dimensions?

$\mathrm{Tr}(I + 4I - 2 \cdot 2I) = 2.$

**P6.** Each token needs 2 ms of compute and moves 1 GB of memory, on a chip with 200 GB/s of bandwidth. Which limit binds?

Memory takes $1/200 = 5$ ms, which exceeds 2 ms, so the step is memory-bound at 5 ms.

**P7.** Which split of a model costs the most inter-device communication each layer?

Tensor parallelism, since each layer splits its matrices across devices.